

How the Two Winning Models Work

A beginner-friendly guide to the training code, JSON configurations, feature engineering, validation choices, and saved model artifacts.

This guide explains how to read and safely modify the repository. For the complete preprocessing and validation record, use the companion [Preprocessing and Validation Protocol](#).

Draft-entry classifier

Ranks Combine participants by whether they will be drafted in the same year.

Holdout AP: 0.8245

Early-minutes regression

Predicts first-three-season NBA minutes after a player has been drafted.

Holdout R^2 : 0.4819

What this guide answers

The project has two different prediction problems. The cleanest way to understand the code is to follow one row from a materialized parquet file, through a JSON configuration and preprocessing pipeline, into a fitted model and a holdout prediction.

- | | |
|---|--|
| 01 The two questions and results | 06 How JSON fields control the code |
| 02 The repository's simple architecture | 07 Frozen winners versus new searches |
| 03 Regression, step by step | 08 How to modify the project safely |
| 04 Classification, step by step | 09 Common problems and fixes |
| 05 Feature engineering in basketball terms | 10 Source map and limitations |

Best way to learn the implementation: read this guide once, then run the two small teaching notebooks. They call the same reusable functions as the command-line scripts: [classification notebook](#) and [regression notebook](#).

Two related questions, two different populations

	Draft-entry classifier	Early-minutes regression
Question	Will this Combine participant be drafted in the same year?	After being drafted, how many regular-season minutes will this player receive in his first three NBA seasons?
Population	Players present in all four collected Combine source rosters, 2009–2022.	Drafted players in the constructed draft table, 2009–2022.
Target	<code>drafted</code> , a 0/1 label from exact player ID plus draft-year membership in official Draft History.	<code>nba_min_3y</code> , summed minutes from seasons beginning in the draft year through draft year + 2.
Inputs	105 numeric + 2 categorical: Combine, NCAA history, age, development, and composites.	48 numeric: draft position, age, NCAA production, development, and composites.
Frozen config	<code>configs/xgboost_draft_classifier_advanced_best.json</code>	<code>configs/xgboost_minutes_advanced_best.json</code>

858

classification rows

838

regression rows

2020–22

shared final holdout years

Classifier result

Average precision: 0.824542

ROC-AUC: 0.750620

Test rows: 202

AP and ROC-AUC are the strongest holdout claims because they evaluate ranking and were frozen before test scoring.

Regression result

R²: 0.481949

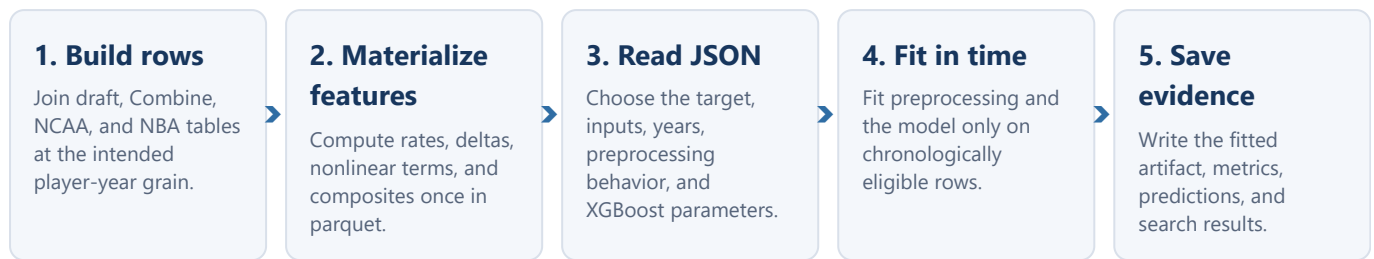
RMSE: 1,568.183 minutes

Test rows: 178

R² says the model explains about 48% of the variation in this later drafted-player cohort.

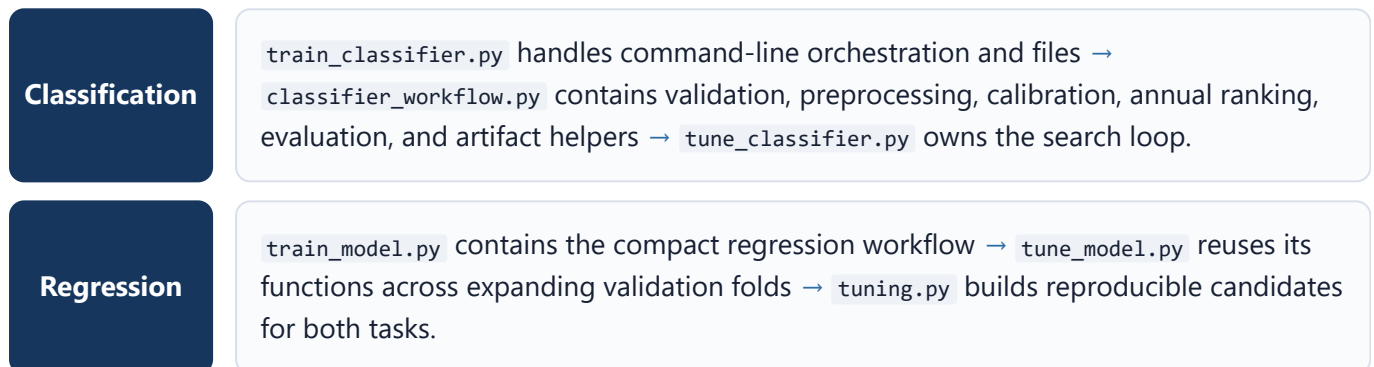
Why draft pick appears in only one model. Draft pick would reveal the answer to “was he drafted?”, so the classifier forbids it. The minutes model begins after the draft and asks about later playing time, so pick number is legitimate information about team investment and expected opportunity.

The code is a small sequence of ordinary steps



For the final frozen run, the estimator is refitted through 2019. Only 2020–2022 remains out of sample.

Where responsibilities live



Why this organization is useful. The command-line scripts read like recipes. Reusable pieces are plain functions, so notebooks and tests can call the exact same logic without duplicating a hidden second implementation.

From a drafted player to a minutes prediction

1. **Load the frozen JSON.** It names the advanced parquet, target, 48 features, chronological split, XGBoost parameters, and output file.
2. **Validate and clean.** Rows missing the target or split year are dropped. Configured numerical columns are coerced to numbers; infinities become `NaN`.
3. **Split by draft year.** The test period is later in time, not a random sample of older players.
4. **Fit preprocessing.** Each numeric column receives a training-fold median, then standardization. The winner has no categorical inputs.
5. **Fit XGBoost.** The target remains raw minutes because `log_transform_target` is `false`.
6. **Refit and evaluate.** The final model fits train + validation, predicts all three named periods, and saves a normal sklearn pipeline with joblib.

```
config = load_config("configs/xgboost_minutes_advanced_best.json")
data = prepare_data(load_data(config["data_path"]), config)
splits = split_data(data, config["split"])

model = make_pipeline(config)
fit_rows = concat(splits["train"], splits["validation"])
model.fit(fit_rows[features], fit_rows["nba_min_3y"])
test_minutes = model.predict(splits["test"][features])
```

Frozen XGBoost settings

Setting	Value	Plain-language role
<code>n_estimators</code> / <code>learning_rate</code>	500 / 0.02	Many small corrections rather than a few aggressive trees.
<code>max_depth</code>	1	Decision stumps keep individual relationships deliberately simple.
<code>min_child_weight</code>	10	Discourages splits supported by very little effective sample weight.
<code>subsample</code> / <code>colsample_bytree</code>	1.0 / 0.8	Use all rows but only 80% of columns for each tree.
<code>reg_alpha</code> / <code>reg_lambda</code>	100 / 25	Strong L1 and L2 regularization limits unstable leaf values.
<code>gamma</code>	200,000	Requires a large squared-error improvement before adding a split. Its scale looks large because the target is raw minutes.

One subtlety: trees do not require standardized inputs, but this project's generic regression pipeline scales them consistently. The scaler and median values are learned only from rows used in each fit, so they do not inspect the holdout.

From a Combine row to a calibrated annual rank

1. **Validate the modeling grain.** The workflow requires one row per `(nba_player_id, draft_year)`, a binary target, and every configured input.
2. **Reject leakage.** IDs, names, draft results, absolute calendar columns, linkage metadata, and post-draft `nba_*` fields cannot be configured as predictors.
3. **Prepare inputs.** Numeric values and `NaN` pass directly to XGBoost. Missing positions become the literal category `missing`, then positions are one-hot encoded.
4. **Fit a calibration model.** A train-only XGBoost model scores 2018–2019; logistic regression maps its log-odds to calibrated probabilities.
5. **Refit the estimator.** XGBoost is fitted through 2019 after removing earlier rows for players who reappear in the 2020–2022 holdout.
6. **Rank within each year.** The frozen operating rule labels the top 63.0137% of the complete participant batch in each year.
7. **Save a bundle.** The joblib artifact contains the estimator, calibrator, decision rule, and feature schema together.

```
config = load_config("configs/xgboost_draft_classifier_advanced_best.json")
data = prepare_data(load_data(resolve(config["data_path"])), config)
splits = chronological_splits(data, config["split"])

model, calibrator, rule, fit_info = fit_configured_classifier(config, splits)
metrics = evaluate(model, splits["test"], features, "drafted",
                  calibrator=calibrator, decision_rule=rule,
                  year_column="draft_year")
```

Frozen XGBoost settings

Setting	Value	Plain-language role
<code>n_estimators</code> / <code>learning_rate</code>	400 / 0.035	Build a conservative score in many small steps.
<code>max_depth</code> / <code>min_child_weight</code>	1 / 3	Use shallow effects and avoid tiny leaves.
<code>subsample</code> / <code>colsample_bytree</code>	0.65 / 0.8	Sample rows and columns to reduce variance.
<code>reg_alpha</code> / <code>reg_lambda</code>	0 / 3	No L1 penalty; moderate L2 shrinkage.
<code>gamma</code> / <code>max_delta_step</code>	0.5 / 1	Demand useful splits and limit abrupt logistic updates.
<code>scale_pos_weight</code>	0.3	Downweights the positive class; chosen on pre-test validation folds.

The annual class label is a batch decision. It only makes sense when scoring the complete Combine-participant set for a year. For one isolated player, use the calibrated probability or ranking score—not the annual top-fraction label.

Turn raw box scores into comparable basketball signals

The main idea is simple: totals reflect opportunity as much as ability. Per-minute rates, efficiency measures, career summaries, and development features make different prospects more comparable while preserving what was actually observed.

Per-40 production

$40 \times \text{statistic} \div \text{minutes}$

Normalizes points, rebounds, assists, steals, blocks, turnovers, and attempts to a common playing-time scale.

both models

Shooting efficiency

$$\text{eFG\%} = (\text{FGM} + 0.5 \times 3\text{PM}) \div \text{FGA}$$
$$\text{TS\%} = \text{PTS} \div [2 \times (\text{FGA} + 0.44 \times \text{FTA})]$$

Credits the extra value of threes and accounts for free throws.

both models

Playmaking and shot profile

$\text{AST}/\text{TOV} \cdot 3\text{PA}/\text{FGA} \cdot \text{FTA}/\text{FGA}$

Describes ball security, perimeter orientation, and ability to reach the line.

both models

Reliable career summaries

sum career totals first, then recompute rates

This weights seasons by their real minutes or attempts instead of averaging small and large samples equally.

both models

Basketball composites

Feature	Formula	Why it exists
Stocks	steals per 40 + blocks per 40	A compact defensive-event measure.
Playmaking margin	assists per 40 – turnovers per 40	Balances creation against lost possessions.
Production index	$\text{PTS} + \text{REB} + \text{AST} + 2(\text{STL} + \text{BLK}) - \text{TOV}$, all per 40	Provides a broad, intentionally transparent production summary.
Usage proxy	$\text{FGA} + 0.44 \times \text{FTA} + \text{TOV}$, all per 40	Approximates possessions a player finishes or turns over.
Shrunk 3P%	$(3\text{PM} + 0.35 \times 40) \div (3\text{PA} + 40)$	Pulls tiny shooting samples toward a fixed 35% prior.
Shrunk FT%	$(\text{FTM} + 0.72 \times 30) \div (\text{FTA} + 30)$	Pulls tiny free-throw samples toward a fixed 72% prior.

Safe division rule. A zero denominator, infinity, or undefined rate becomes missing—not zero. “No usable attempt sample” is different from “made 0%,” and the model should keep that distinction.

Age, development, Combine traits, and draft capital

Age and upside
centered age = age – 21
youth score = 23 – age
age-adjusted output = youth × production
Exposes nonlinear age effects and the interaction between current output and development runway.

Combine-derived traits
wingspan – height · standing reach – height
max reach = standing reach + max vertical
BMI = 703 × weight ÷ height²
Separates functional length, explosiveness, and mass-for-height from raw measurements.

Recent development
classifier delta = last season – previous season
regression difference = last season – career
regression ratio = last season ÷ career
Captures direction of change. The three definitions are related but not interchangeable.

Nonlinear draft capital
 $\log(1 + \text{pick}) \cdot \sqrt{\text{pick}} \cdot 1/\text{pick} \cdot \text{pick}^2$
 $\text{pick} \leq 5 \cdot 10 \cdot 14 \cdot 30$
Represents the steep, nonlinear connection between team investment and early NBA opportunity.

regression only

Exactly what the winning feature sets contain

Model	Family	Count	Examples
Classifier	Anthropometric	8	Height, weight, wingspan, standing reach, body fat, hand dimensions.
	Athletic drills	6	Vertical leaps, agility, sprint, bench press.
	Combine-derived	5	Length differences, maximum reach, vertical difference, BMI.
	Last NCAA season	32	Volume, efficiency, per-40 rates, and composites.
	NCAA career	32	Career totals, recomputed rates, and composites.
	Development + age	22	16 last-vs-previous signals and 6 age/upside signals.
	Categorical	2	position and college_position .
Regression	Base pick, age, college	17	Pick, age, recent NCAA output, seven last-minus-career differences.
	Draft-capital transforms	8	Four continuous transforms and four pick bands.
	Age and interactions	6	Centered/squared age, youth, age-adjusted output, pick × age.
	Production composites	8	Shrunk shooting, stocks, playmaking, and production.
	Last-to-career ratios	9	Scale-aware recent form relative to career level.

Materialized means already computed. The winning training scripts do not calculate these formulas on every run; the columns already exist in `draft_classification_advanced.parquet` and `drafted_players_advanced.parquet`. The JSON `feature_engineering` block in the minutes config records provenance. To add or change a formula, rebuild the parquet before adding the new column name to a config.

How JSON fields control the Python code

JSON field	What it controls	Where it is consumed
<code>data_path</code>	The parquet or CSV modeling table.	<code>train_model.py</code> or <code>classifier_workflow.py</code> loaders.
<code>target</code>	The column the estimator learns to predict.	Preparation, fitting, and metric functions.
<code>numerical_features</code> <code>categorical_features</code>	The exact model schema and order.	The sklearn <code>ColumnTransformer</code> .
<code>split</code>	Year column and train/validation/test boundaries.	<code>split_data</code> or <code>chronological_splits</code> .
<code>model.name</code> <code>model.hyperparameters</code>	Estimator family and constructor arguments.	<code>build_estimator</code> or <code>make_model</code> .
<code>refit_on_train_validation</code>	Whether the final estimator learns from data through 2019.	Both direct-training workflows.
<code>log_transform_target</code>	Whether regression fits <code>log1p(target)</code> .	<code>train_model.make_pipeline</code> ; false for the winner.
<code>numerical_missing_strategy</code>	Median imputation or native missing values.	<code>classifier_workflow.make_pipeline</code> ; native for the winner.
<code>probability_calibration</code>	Optional sigmoid map learned on validation.	<code>fit_probability_calibrator</code> .
<code>group_column</code>	Player identity used to remove cross-period overlap.	<code>purge_group_overlap</code> .
<code>decision_rule_fitted</code>	The frozen probability threshold or annual ranking fraction.	<code>fit_configured_classifier</code> and <code>evaluate</code> .
<code>tuning</code>	Search space, validation folds, fixed candidates, seed, and search-only output paths.	<code>tune_model.py</code> or <code>tune_classifier.py</code> , not ordinary fitting.
<code>output_model_path</code> <code>report_path</code>	Direct frozen-run artifacts.	The two command-line training scripts.

Documentation fields are intentional. Descriptions, evaluation notes, and feature-selection provenance do not change XGBoost. They travel with the experiment so a future reader can understand what was selected, when it was selected, and which claims are clean holdout evidence.

Keep frozen winners separate from new searches

Frozen reference

A checked, named configuration whose parameters and reported holdout result should not change.

```
*_advanced_best.json
```

Use it for reproduction, the toy notebooks, and comparison.

Search output

A new experiment produced by tuning. It may be better or worse and needs its own filenames.

```
*_advanced_search_best.json
```

Inspect validation evidence before deciding whether to promote it.

What each tuner now does

	Regression tuner	Classification tuner
Validation design	Three expanding windows: 2014–15, 2016–17, 2018–19.	Six expanding annual folds: 2012 through 2017.
Selection score	Pooled out-of-fold RMSE; lower is better.	Mean annual normalized AP; higher is better.
Candidate source	Configured fixed candidates plus a reproducible sample from the search lists.	Configured fixed candidates plus a reproducible sample from the search lists.
Leakage control	Every fold ends before its validation years; test years are rejected.	Same chronological control, plus repeated-player purging.
Protection	Both scripts refuse to write <code>best_config_path</code>	

For classification year y : **normalized AP** = $(AP_y - \text{prevalence}_y) \div (1 - \text{prevalence}_y)$. This gives random/prevalence-level ranking a baseline of zero and a perfect ranking a score of one before averaging years equally.

```
# Reproduce the frozen winners
python src/train_classifier.py --config configs/xgboost_draft_classifier_advanced_best.json
python src/train_model.py --config configs/xgboost_minutes_advanced_best.json

# Run small, separate search experiments
python src/tune_classifier.py --config configs/xgboost_draft_classifier_advanced_best.json --n-iter 5
python src/tune_model.py --config configs/xgboost_minutes_advanced_best.json --n-iter 5
```

A quick search does not redefine “best.” The `--n-iter` flag temporarily samples fewer candidates for a smoke test. Its output is deliberately written to separate `search` filenames and should not replace the historical winner merely because the script completed.

Common changes and the smallest correct workflow

You want to...	Change	Then verify
Try different XGBoost settings	Copy a config, give its model/results/best-config outputs new names, edit <code>tuning.search_space</code> , and run the tuner.	Compare pre-test validation first. Treat 2020–2022 as confirmation, not the selector.
Add an existing parquet column	Add its exact name to the numerical or categorical list in an experiment config.	Confirm timing and leakage safety; rerun chronological tuning.
Create a new feature formula	Implement it in the appropriate dataset builder/materializer, rebuild the advanced parquet, then configure it.	Check undefined denominators, missingness, units, and whether the formula uses only information available at prediction time.
Switch estimator families	Regression accepts Ridge/XGBoost; classification accepts logistic regression/XGBoost.	For logistic classification, use median imputation because logistic regression cannot consume native NaN.
Move the evaluation years	Update the split only after confirming the parquet contains those years.	Keep a genuinely later, untouched final period and rebuild all validation folds.
Score new classifier rows	Use the saved bundle's schema, estimator, and calibrator.	Only apply the annual class rule to a complete yearly participant batch.

A good experiment checklist

- ✓ Copy the frozen config instead of editing it in place.
- ✓ Choose new output filenames before running anything.
- ✓ Write one sentence describing the hypothesis behind the change.
- ✓ Keep target, population, and split fixed when comparing feature or parameter choices.
- ✓ Use validation to choose; inspect test only after the choice is frozen.
- ✓ Save the resulting config and metrics together.
- ✓ Run `python -m unittest discover -s tests -v`.

Human-readable code habit: comments should explain why a step exists—leakage prevention, timing, missingness, or basketball meaning. The function and variable names should explain what the code mechanically does.

When something goes wrong, start with the contract

Symptom	Likely cause	What to inspect
"Columns not found" or "Dataset is missing columns"	The JSON and parquet schemas do not match.	Spell the feature exactly, verify <code>data_path</code> , and rebuild the advanced table if the feature is new.
"Outcome or identity fields cannot be features"	The classifier leakage guard found a target, identity, calendar, linkage, or post-draft field.	Remove it. Do not bypass the guard to improve a metric.
Duplicate player-year error	The classification table no longer has one row per event.	Fix the upstream join or aggregation; do not silently drop an arbitrary duplicate.
One chronological split is empty	The configured years are absent or ordered incorrectly.	Inspect <code>draft_year</code> coverage and the five split boundaries.
AP or R^2 suddenly changed	A search output was mistaken for the frozen config, data changed, or a different split/model artifact was loaded.	Compare hyperparameters and filenames first; then confirm row counts and dependency versions.
<code>.predict()</code> fails on classifier joblib	The advanced classifier is a dictionary bundle, not a bare pipeline.	Use <code>artifact["estimator"]</code> , its calibrator, and its decision rule—or reuse workflow helpers.
A classifier label changes when other rows are added	The annual top-fraction rule ranks a batch.	This is expected. Probabilities are row-level; annual labels are cohort-level.
Missing values look like zeros	A custom preprocessing change may have destroyed missingness semantics.	Numeric absence should be <code>NaN</code> ; only categorical absence becomes the string <code>missing</code> .

Saved artifact anatomy

Regression joblib

A fitted sklearn `Pipeline` containing the numeric preprocessor and XGBoost regressor.

```
model = joblib.load(path)
minutes = model.predict(rows[features])
```

Classification joblib

A versioned dictionary containing estimator, sigmoid calibrator, decision rule, target, features, and training cutoff.

```
artifact = joblib.load(path)
model = artifact["estimator"]
rule = artifact["decision_rule"]
```

Where to read next

Question	Primary source
How does direct classifier training run?	<code>src/train_classifier.py</code>
Where are classifier preparation, leakage checks, calibration, annual decisions, metrics, and artifacts?	<code>src/classifier_workflow.py</code>
How are classifier parameters compared?	<code>src/tune_classifier.py</code> · <code>src/tuning.py</code>
How does regression preparation and fitting work?	<code>src/train_model.py</code>
How are regression parameters compared across expanding folds?	<code>src/tune_model.py</code> · <code>src/tuning.py</code>
How is the base Combine classification cohort and exact target built?	<code>nba_draft_data_collection/build_draft_classification_dataset.py</code>
How are NCAA histories, rates, development, and Combine composites added?	<code>nba_draft_data_collection/build_enriched_draft_classification_dataset.py</code>
How are drafted-player college histories and the three-year minutes target built?	<code>nba_draft_data_collection/build_model_dataset_v3.py</code>
Where is advanced materialization provenance?	<code>.archive/src/build_advanced_model_datasets.py</code> · <code>.archive/src/advanced_feature_engineering.py</code> · <code>.archive/src/feature_engineering.py</code>
Where are the frozen schemas and hyperparameters?	<code>configs/xgboost_draft_classifier_advanced_best.json</code> · <code>configs/xgboost_minutes_advanced_best.json</code>

Interactive walkthroughs

Train the best classification model

Loads the frozen config, fits the same calibrated workflow, reports holdout metrics and a precision–recall curve, and inspects importance.

Train the best regression model

Loads the frozen config, shows preprocessing, fits once, evaluates the holdout, plots residuals, and inspects example predictions.

Interpretation boundaries

- The classifier describes players who reached this consistently observed Combine cohort, not every NCAA player.
- The regression describes drafted players and deliberately includes draft position. It is not a talent-only or fully pre-draft model.

- Predictions are associations, not estimates of what would happen if a team changed a player's draft position, role, or development plan.
- Unresolved NCAA histories remain missing. Native missing handling preserves the gap but cannot eliminate selection bias in who links successfully.
- Multiple experiments have been viewed on the 2020–2022 period. A future cohort is the appropriate next independent confirmation.
- For classification, AP and ROC-AUC are the clean frozen ranking claims. The repaired annual decision rule and probability calibration deserve confirmation on future data.

Companion reference: the [Preprocessing and Validation Protocol](#) contains the exhaustive feature inventories, linkage rules, missing-value treatment, validation evidence, and methodological caveats. This guide is the shorter map for understanding and changing the code.

Prepared from the checked-in code, frozen configurations, advanced parquet schemas, archived experiment reports, and refactored classifier workflow in `D:\Mestrado\nba-draft`. All links are relative so the HTML and rendered PDF can remain with the repository.